Blockchain A4

My Github: https://github.com/sakhilebut5-star/Blockchain-Assignment-4

Introduction

This lab will demonstrate the core components of a cryptocurrency-style wallet and digital signature workflow. We create a wallet system for two people (Alice and Bob) using ECDSA on the secp256k1 elliptic curve, this is the same curve used used by Bitcoin and Ethereum.

Key generation and wallet addresses

Each wallet instance generates its own ECDSA private key (a random 256-bit scalar) on secp256k1 using the ECDSA library on python. Then the corresponding public key is derived.

Address derivation

The address is derived from the public key using a simplified version of the Bitcoin P2PKH recipe:

1. Take the raw public key bytes: 0x04 concatenated with the X and Y coordinates (uncompressed SEC1 encoding, 65 bytes).
2. Compute SHA-256 of the public key bytes.
3. Compute RIPEMD-160 of that SHA-256 output → a 20-byte "public key hash."
4. Prepend a 1-byte version prefix (0x00, a lab-only marker, not a registered Bitcoin mainnet byte).
5. Compute SHA-256(SHA-256(versioned payload)) and take the first 4 bytes as a checksum (Base58Check pattern).
6. Append the checksum to the versioned payload and Base58-encode the result.

The resulting addresses are:

Alice: 1KeJyNTpYSivCpWDtkgXNfHpsBV1qaeoAA

Bob: 1PWet2z53BwWF1M6RHXBKrKRs5j2xEs97w

Signing, verification, and tamper detection

A transaction payload is a JSON-serialisable dictionary (serialised deterministically before signing so that both parties hash the exact same bytes, as shown below:

```
tx = {
    "from": alice.address,
    "to": bob.address,
    "amount": 50.0,
    "currency": "ZAR-remit",
    "nonce": 1,
}
```

The amount field was then changed from 50.0 to 5000.0 after signing, and the original signature was re-checked against both the tampered payload and against Bob's public key.

Check	Result
Alice's signature verified against the original payload, using Alice's public key	True
Same signature verified against the original payload, using Bob's public key instead of Alice's	False, proves authenticity
Same signature verified against the payload with 'amount' changed from 50.0 to 5000.0	False, proves integrity

These results confirm the two properties a digital signature provides: authenticity (only the private key holder could have produced a signature that verifies under the corresponding public key) and integrity (any change to the signed data invalidates the signature).

Conclusion

Two distinct secp256k1 ECDSA key pairs were generated, addresses were derived using a fully documented sequence of hashing and encoding operations, and a transaction was successfully signed and verified. Verification correctly failed when the signature was checked against Bob's public key and when the transaction amount was changed. These results demonstrate the authenticity and integrity properties of digital signatures and highlight why private-key custody is central to the security of a remittance wallet.